

E-OTEL REZERVASYON YÖNETİM SİSTEMİ

Teknik Rapor

Ders	Veri Yapıları
Proje	Hotel Reservation System
Dil	Java (Swing GUI)
Tarih	18.05.2026
Hazırlayanlar	Arnta Moumin Mert Ülker Soydan Küçük

. İÇİNDEKİLER

1.	Giriş	3
1.1	Problem Tanımı	3
1.2	Motivasyon	3
1.3	Proje Kapsamı	3
2.	Sistem Mimarisi	4
2.1	Genel Yapı	4
2.2	Katmanlar	4
3.	Kullanılan Veri Yapıları	5
3.1	HashMap — O(1) Erişim	5
3.2	Custom BST — Sıralı Geçmiş	5
3.3	LinkedList — Bekleme Kuyruğu	6
3.4	Interval Çarpışma Kontrolü	6
4.	Temel Sınıflar ve Yöntemler	7
4.1	Veri Modelleri	7
4.2	İş Mantığı Metodları	8
5.	Karmaşıklık Analizi	9
5.1	Zaman Karmaşıklığı	9
5.2	Alan Karmaşıklığı	9
6.	Test Senaryoları	10
6.1	Normal Durum Testleri	10
6.2	Sınır Değer Testleri	11
6.3	Hata Durumu Testleri	11
7.	Sonuç	12
7.1	Karşılaşılan Zorluklar	12
7.2	Öğrenilenler	12
7.3	Geliştirme Fikirleri	12
8.	Kaynakça	13

1. GİRİŞ

1.1 Problem Tanımı

Otel rezervasyon sistemlerinde en büyük sorun çakışan rezervasyonları engellemek, bekleme listesini yönetmek ve geçmiş kayıtlara hızlı erişmektir. Bu proje, bu sorunları veri yapıları kullanarak çözmeyi amaçlamaktadır.

1.2 Motivasyon

Derste öğrenilen BST, HashMap ve LinkedList yapılarını gerçek bir projede kullanmak istedik. Her veri yapısını belirli bir problemi çözmek için seçtik ve bunun performansa etkisini analiz ettik.

1.3 Proje Kapsamı

Sistem aşağıdaki temel işlevleri kapsamaktadır:

- Çok şubeli otel rezervasyon yönetimi
- Tarih aralığı çarpışma kontrolü ile müsaitlik sorgulama
- FIFO tabanlı bekleme listesi yönetimi
- BST ile tarihe göre sıralı geçmiş rezervasyon görüntüleme
- CSV tabanlı kalıcı veri saklama (harici kütüphane kullanılmadan)
- Java Swing ile masaüstü grafik arayüzü

2. SİSTEM MİMARİSİ

2.1 Genel Yapı

Uygulama tek bir Java sınıfı içinde iç içe statik sınıflarla organize edilmiştir. Tüm veri yapıları statik alanlar olarak tanımlanmış, tek bir uygulama örneği üzerinden erişilmektedir.

2.2 Katmanlar

Katman	Sınıf / Bileşen	Sorumluluk
Veri Modeli	Room, Customer, Reservation	Temel nesneleri ve alanlarını tanımlar
Veri Yapıları	BST, HashMap, LinkedList, ArrayList	Verileri depolar ve erişim sağlar
İş Mantığı	handleBooking, doCheckout, doCancel	Rezervasyon akışını kontrol eder
Kalıcılık	saveData, loadData	CSV'ye yazar / okur
Arayüz	buildUI, buildXxxTab	Swing sekme ve tablolarını oluşturur

Sınıf İlişkileri (UML Özeti)

Sınıf	Alanlar	İlişki
Room	id, number, type, branch, intervals:List<long[]>	1 Room — N Reservation (roomId üzerinden)
Customer	id, name, email	1 Customer — N Reservation (custId üzerinden)
Reservation	id, roomId, custId, in, out, status	Orta tablo: Room ve Customer'ı ilişkilendirir
BST	root:Node (iç sınıf)	BST → Node (kompozisyon), Reservation depolar

3. KULLANILAN VERİ YAPILARI

3.1 HashMap — O(1) Hızında Erişim

HashMap, anahtar-değer çiftlerini hash tablosuyla depolar ve O(1) hızında erişim sağlar. ArrayList kullansaydık belirli bir odayı bulmak için O(n) tarama yapmak gerekirdi. Sistemde iki HashMap bulunur:

- rooms — Oda ID'sini Room nesnesine eşler (örn: "R1" → Room{101, Standart, Şube A})
- custs — Müşteri ID'sini Customer nesnesine eşler

Örnek kullanım kodundan alıntı:

```
Room room = rooms.get(r.roomId); // O(1) - ID ile direkt erişim
```

3.2 Custom BST — Tarihe Göre Sıralı Geçmiş

Java'nın hazır TreeMap/TreeSet sınıfları kullanılmadan sıfırdan yazılan BST, tamamlanan rezervasyonları giriş tarihine göre sıralı tutar. Inorder (Sol-Kök-Sağ) gezinme her zaman sıralı sonuç ürettiğinden geçmiş sekmesi ek sıralama yapmadan düzenli görünür.

BST düğüm ekleme mantığı:

```
if (r.in.isBefore(n.d.in)) → sol dala git (daha eski tarih) else → sağ dala git (daha yeni veya eşit tarih)
```

Ortalama durumda ekleme O(log n) sürer. Veriler sıralı gelirse ağaç dengesizleşerek O(n)'e düşebilir.

3.3 LinkedList — FIFO Bekleme Kuyruğu

Bekleme listesi FIFO (ilk gelen ilk çıkar) mantığıyla çalışır. LinkedList başa/sona ekleme ve çıkarma işlemlerini O(1)'de yapar. ArrayList kullanılsaydı baş eleman silme O(n) olurdu.

- Oda dolu → waitlist.add(res) — listenin sonuna O(1) ekleme
- Oda boşalınca → iterator ile tarama, uygun rezervasyon bulundu → it.remove() ile O(1) silme

3.4 Interval (Aralık) Çarpışma Kontrolü

Her oda rezervasyonlarını [giriş_günü, çıkış_günü] çiftleri olarak saklar. Tarihler epoch-day formatına (1 Ocak 1970'den geçen gün sayısı) dönüştürülür; böylece karşılaştırma basit sayı karşılaştırmasına indirgenir.

Çarpışma formülü:

```
İki aralık [a, b) ve [c, d) ÇARPIŞIYOR ise: a < d VE c < b Örnek 1 (çarpışma): [3,7) n [5,9) → 3<9 ✓ ve 5<7 ✓ → DOLU Örnek 2 (serbest): [3,5) n [6,9) → 3<9 ✓ ama 6<5 ✗ → MÜSAİT
```

Zaman karmaşıklığı: O(n) — n: odanın mevcut rezervasyon sayısı

4. TEMEL SINIFLAR VE YÖNTEMLER

4.1 Veri Modeli Sınıfları

Sınıf	Alanlar	Açıklama
Room	id, number, type, branch, intervals	Oda bilgilerini ve tarih aralıklarını tutar. isAvailable(), book(), release() metodları içerir.
Customer	id, name, email	Müşteri kimlik ve iletişim bilgilerini depolar.
Reservation	id, roomId, custId, in, out, status	Rezervasyon bağlantısını kurar. status: ACTIVE WAITLIST COMPLETED IPTAL
BST	root (Node)	insert(Reservation), inorder() metodlarını içeren özel ikili arama ağacı.

4.2 İş Mantığı Metodları

Metod	İşlev	Kullandığı Veri Yapısı
handleBooking()	Yeni rezervasyon oluşturur. Oda müsaitse active'e ekler, değilse waitlist'e atar.	HashMap, LinkedList, ArrayList, Interval
doCheckout()	Aktif rezervasyonu tamamlar, odayı serbest bırakır, waitlist'i kontrol eder.	BST (insert), LinkedList (remove)
doCancelById()	Rezervasyonu iptal eder, oda aralığını siler, BST geçmişine ekler.	ArrayList, BST, HashMap
room.isAvailable()	Belirtilen tarih aralığı için odanın müsait olup olmadığını kontrol eder.	Interval Collision (O(n))
history.inorder()	Tamamlanan rezervasyonları giriş tarihine göre sıralı döndürür.	BST (O(n) gezinme)
saveData()	Tüm verileri CSV dosyasına yazar. Program kapanışında otomatik çalışır.	Tüm yapılar (O(n) tarama)
loadData()	CSV'den veri okuyarak bellek yapılarını yeniden oluşturur.	HashMap, BST, LinkedList, ArrayList

5. KARMAŞIKLIK ANALİZİ

5.1 Zaman Karmaşıklığı

İşlem	Kullanılan Yapı	Ortalama	En Kötü	Açıklama
Oda/Müşteri getir	HashMap	$O(1)$	$O(n)$	Hash çakışması olmadığında $O(1)$
Müsaitlik kontrolü	Interval List	$O(k)$	$O(k)$	k = odanın rezervasyon sayısı
BST ekleme	BST	$O(\log n)$	$O(n)$	Dengeli ağaçta $O(\log n)$
BST inorder	BST	$O(n)$	$O(n)$	Tüm düğümler bir kez ziyaret edilir
Bekleme ekleme	LinkedList	$O(1)$	$O(1)$	Sona ekleme her zaman sabit
Bekleme silme	LinkedList	$O(w)$	$O(w)$	w = bekleme listesi uzunluğu
CSV kaydetme	Tümü	$O(n)$	$O(n)$	n = toplam kayıt sayısı

5.2 Alan Karmaşıklığı

Yapı	Alan Karmaşıklığı	Açıklama
HashMap (rooms)	$O(r)$	r = toplam oda sayısı
HashMap (custs)	$O(c)$	c = toplam müşteri sayısı
BST (history)	$O(h)$	h = tamamlanan + iptal rezervasyon sayısı
LinkedList (waitlist)	$O(w)$	w = bekleme listesindeki rezervasyon sayısı
ArrayList (active)	$O(a)$	a = anlık aktif rezervasyon sayısı
TOPLAM	$O(r + c + h + w + a)$	Sistemdeki toplam nesne sayısı ile doğrusal

6. TEST SENARYOLARI

6.1 Normal Durum Testleri

Test No	Test Adı	Girdi	Beklenen Çıktı	Sonuç
T-01	Başarılı Rezervasyon	Oda 101, Giriş: 01-07-2026 Çıkış: 05-07-2026	Rezervasyon ACTIVE, oda dolu olarak işaretlenir	✓ Geçti
T-02	Oda Checkout	Aktif rezervasyon seçilip 'Çıkış Yap' tıklanır	Oda müsait olur, rezervasyon BST'ye eklenir	✓ Geçti
T-03	Geçmiş Sıralama	3 farklı tarihte rezervasyon tamamlanır	Geçmiş sekmesi tarihe göre küçükten büyüğe sıralı gösterir	✓ Geçti
T-04	Bekleme → Aktif	Dolu odaya rezervasyon, ardından mevcut checkout	Bekleme listesindeki rezervasyon otomatik aktif olur	✓ Geçti

6.2 Sınır Değer Testleri

Test No	Test Adı	Girdi	Beklenen Çıktı	Sonuç
T-05	1 Gecelik Konaklama	Giriş: 10-08-2026 Çıkış: 11-08-2026	Geçerli rezervasyon (minimum süre)	✓ Geçti
T-06	Tam Bitişik Tarihler	Res1: [01-07, 05-07) Res2: [05-07, 10-07)	Çarpışma YOK — oda her iki rezervasyon için de müsait	✓ Geçti
T-07	Tam Örtüşen Tarihler	Aynı oda, aynı tarih aralığı [01-07, 10-07) tekrar	Çarpışma VAR — bekleme listesine eklenir	✓ Geçti
T-08	Kısmi Örtüşme	Res1: [01-07, 08-07) Res2: [05-07, 12-07)	Çarpışma VAR (3 gün örtüşme) — bekleme listesine	✓ Geçti

6.3 Hata Durumu Testleri

Test No	Test Adı	Girdi	Beklenen Çıktı	Sonuç
T-09	Geçersiz Tarih Formatı	Giriş: '2026/07/01' (yanlış format)	'Geçersiz tarih formatı. gg-AA-yyyy kullanın' uyarısı	✓ Geçti
T-10	Çıkış < Giriş	Giriş: 10-07-2026 Çıkış: 05-07-2026	'Çıkış tarihi giriş tarihinden sonra olmalıdır' uyarısı	✓ Geçti
T-11	Boş Alan	Oda numarası alanı boş bırakılır	'Şube hariç tüm alanlar zorunludur' uyarısı	✓ Geçti

Test No	Test Adı	Girdi	Beklenen Çıktı	Sonuç
T-12	Olmayan Oda	Oda No: '999' (kayıtlı değil)	'Oda bulunamadı. Oda numarası / şubeyi kontrol edin' uyarısı	✓ Geçti

7. SONUÇ

7.1 Karşılaşılan Zorluklar

- BST'yi sıfırdan yazmak: TreeMap kullanımı yasak olduğundan insert ve inorder metodlarını rekürsiyon ile yazmak başta kafa karıştırıcıydı. Çağrı yığınına kağıda çizerek çözdük.
- Interval Çarpışma Formülü: ' $a < d$ VE $c < b$ ' formülünü türetmek ve bitişik tarihler gibi köşe durumlarını doğru yönetmek dikkat gerektirdi.
- Bekleme Listesi Senkronizasyonu: Oda boşaldığında doğru bekleme kaydını aktif etmek için iterator kullanımı zorunlu oldu.
- CSV Kalıcılığı: Harici kütüphane kullanmadan farklı statüdeki rezervasyonları (ACTIVE, WAITLIST, COMPLETED, IPTAL) doğru ayırtmak özen gerektirdi.

7.2 Öğrenilenler

- Her veri yapısının farklı bir avantajı var; 'tek doğru' yapı diye bir şey yok, probleme göre seçmek gerekiyor.
- BST inorder gezinmesi her zaman sıralı sonuç üretir — bu özellik pratikte çok işe yarıyor.
- HashMap'in $O(1)$ erişimi, büyük veri setlerinde ArrayList'e göre ciddi fark yaratıyor.
- Epoch-day dönüşümü gibi küçük bir optimizasyon tarih karşılaştırmasını büyük ölçüde basitleştiriyor.
- Swing ile GUI geliştirmenin temellerini ve olay güdümlü programlamayı öğrendik.

7.3 Geliştirme Fikirleri

- AVL / Kırmızı-Siyah Ağaç: Mevcut BST sıralı veri geldiğinde $O(n)$ 'e düşer. Öz-dengeleyici bir yapıyla bu sorun ortadan kalkar.
- Gerçek Interval Tree: Müsaitlik kontrolü şu an $O(k)$. Interval tree ile $O(\log n + m)$ 'ye indirilebilir.
- Veritabanı: CSV yerine SQLite veya H2 kullanılarak veri tutarlılığı artırılabilir.
- Fiyatlandırma: Oda tipi ve sezona göre dinamik fiyat hesaplama eklenebilir.
- E-posta Bildirimi: Rezervasyon onayı ve hatırlatma için e-posta entegre edilebilir.

8. KAYNAKÇA

- [1] Ders Notları. Veri Yapıları ve Algoritmalar — Ders Slaytları. (BST, HashMap, LinkedList ve karmaşıklık analizine ait ders materyalleri)
- [2] Oracle. (2024). Java SE Documentation — Collections Framework. <https://docs.oracle.com/en/java/index.html> (HashMap, LinkedList, ArrayList API belgeleri)
- [3] Oracle. (2024). Java SE Documentation — Swing Tutorial. <https://docs.oracle.com/javase/tutorial/uiswing/> (JFrame, JTabbedPane, JTable bileşenleri)
- [4] GeeksForGeeks. (2024). Binary Search Tree — Insert and Inorder Traversal. <https://www.geeksforgeeks.org/binary-search-tree-data-structure/> (BST ekleme ve gezinme algoritması)
- [5] W3Schools. (2024). Java Data Structures. https://www.w3schools.com/java/java_linkedlist.asp (LinkedList ve ArrayList kullanım örnekleri)